

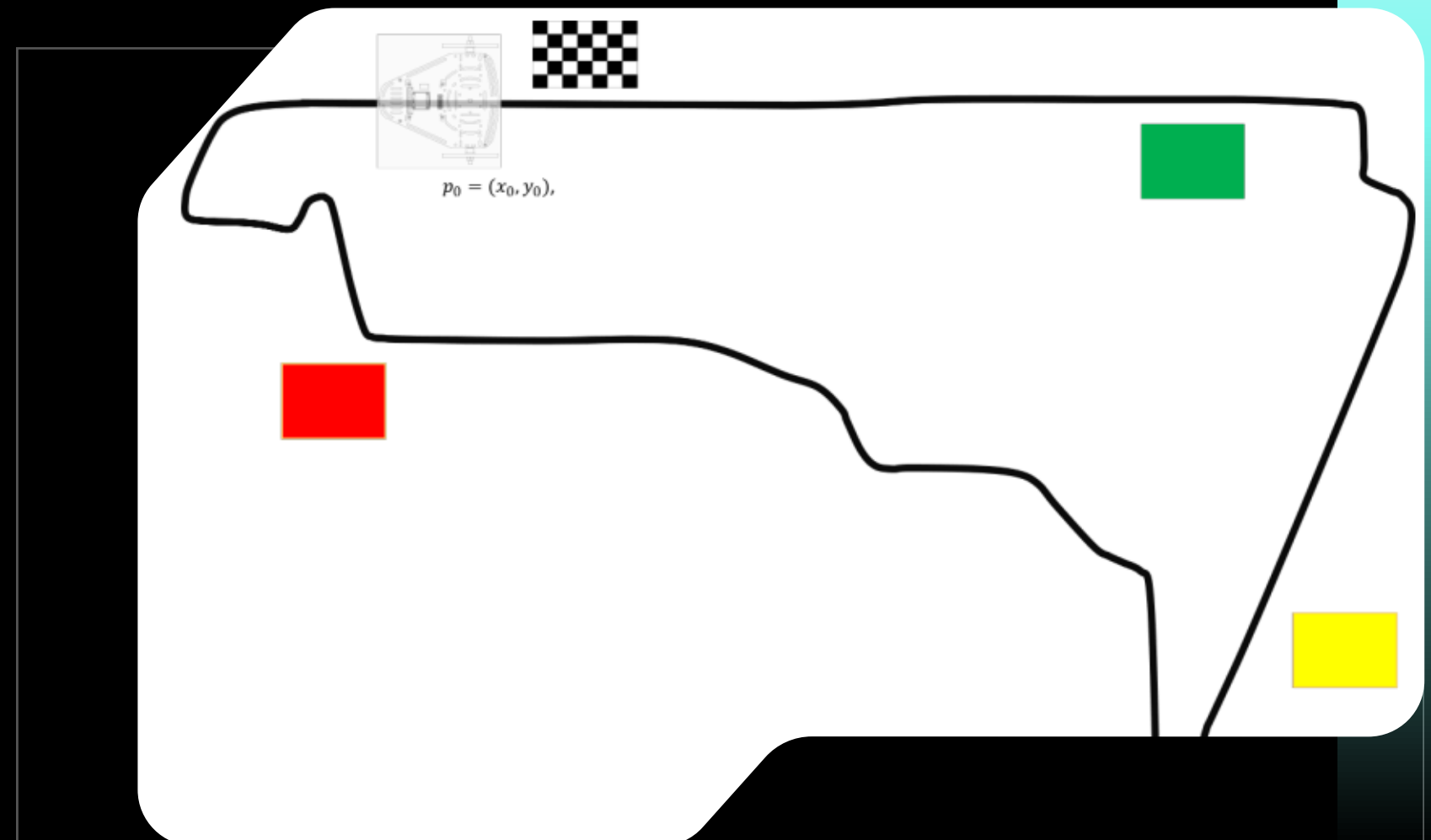


# SEGUIDOR DE LINEA

OMAR PÉREZ DOMÍNGUEZ  
EZEQUIEL LUNA TREJO  
ANTONIO MENDEZ RODRIGUEZ

# RETO

Implementación de un  
seguidor de línea robusto  
con detección de  
semáforos en Puzzlebot



# VISIÓN (SEGUIDOR DE LINEA)

- Umbralizacion adaptativa

Es una técnica de procesamiento de imágenes que permite segmentar un objeto del fondo, incluso cuando la iluminación no es uniforme. A diferencia del método tradicional, que usa un único valor para toda la imagen, este método calcula un umbral diferente para cada pequeño sector.

- Filtro gaussiano

Es una técnica de suavizado que reduce el ruido y los detalles innecesarios de la imagen (como texturas del piso o granos de la cámara). Aplica un desenfoque matemático que ayuda a que los algoritmos de detección se centren solo en las formas grandes y claras, como la línea de la pista.

# VISIÓN (SEGUIDOR DE LINEA)

- Segmentacion por ventanas deslizables

Consiste en dividir la zona de interés de la imagen en varias franjas horizontales independientes (Se divide en 6 ventanas) En lugar de analizar toda la imagen de golpe, el robot busca la línea sección por sección, lo que permite seguir el camino de manera más precisa incluso si la línea se curva o desaparece parcialmente.

- Múltiples centroides

A partir de las ventanas deslizantes, se calcula el centro de masa de los píxeles blancos en cada franja. Al unir estos puntos, el robot genera una representación geométrica de la trayectoria. Esto permite no solo saber dónde está la línea ahora, sino predecir hacia dónde se dirige (curvatura) para ajustar la velocidad.

# NODO CONTROLLER

Este nodo actúa como el centro de datos. Para que el robot se mueva necesitamos sincronizar tres fuentes de información:

- Visión: Para detectar la línea y el semáforo.
- Odometría: Para conocer la orientación real del robot.
- Actuación: Publicamos en `cmd_vel` para mover los motores.

El Algoritmo de Control (PI + Anti-Windup)

- Implementamos un Controlador Proporcional-Integral (PI)
- Término Proporcional ( $K_p$ ): Corrige el error en el instante.
- Término Integral ( $K_i$ ): Acumula el error histórico para eliminar desviaciones constantes en rectas.
- Anti-Windup: Limitamos la integral (`max_integral`) para evitar que el robot "sobre-reaccione" al salir de curvas cerradas..

# FORMULAS

## 1. En el Nodo de Visión: Detección de la Línea

Para explicar cómo el robot sabe dónde está la línea, usa la fórmula de los Momentos de Imagen (Centroides).

- Fórmula del Centroide ( $C_x$ ):
- $C_x = M_{10} / M_{00}$ 
  - Explicación:  $M_{10}$  es la suma de la posición de los píxeles blancos y  $M_{00}$  es el área total (cuántos píxeles blancos hay). Esto nos da el centro de la línea en la pantalla.
- Fórmula del Error Lateral ( $e$ ):
- $e = \text{Centro de imagen} - C_x$ 
  - Explicación: Si el error es positivo, el robot está a la derecha; si es negativo, está a la izquierda.

## 2. Nodo de Control: Algoritmo PI

Es el nodo decide cuánto debe girar el volante para corregir el camino.

- Ecuación del Controlador PI:
- $\text{Giro} = (K_p * \text{Error}) + (K_i * \text{Error Acumulado})$
- Parte Proporcional ( $K_p * \text{Error}$ ): Corrige el error actual. Si el error es grande, el giro es fuerte. Es la reacción inmediata.
- Parte Integral ( $K_i * \text{Error Acumulado}$ ): Corrige errores que se mantienen en el tiempo (por ejemplo, si el robot se queda un poquito desviado hacia un lado).
- Anti-Windup: Es un límite de seguridad que evita que el "Error Acumulado" crezca demasiado y afecte al robot.

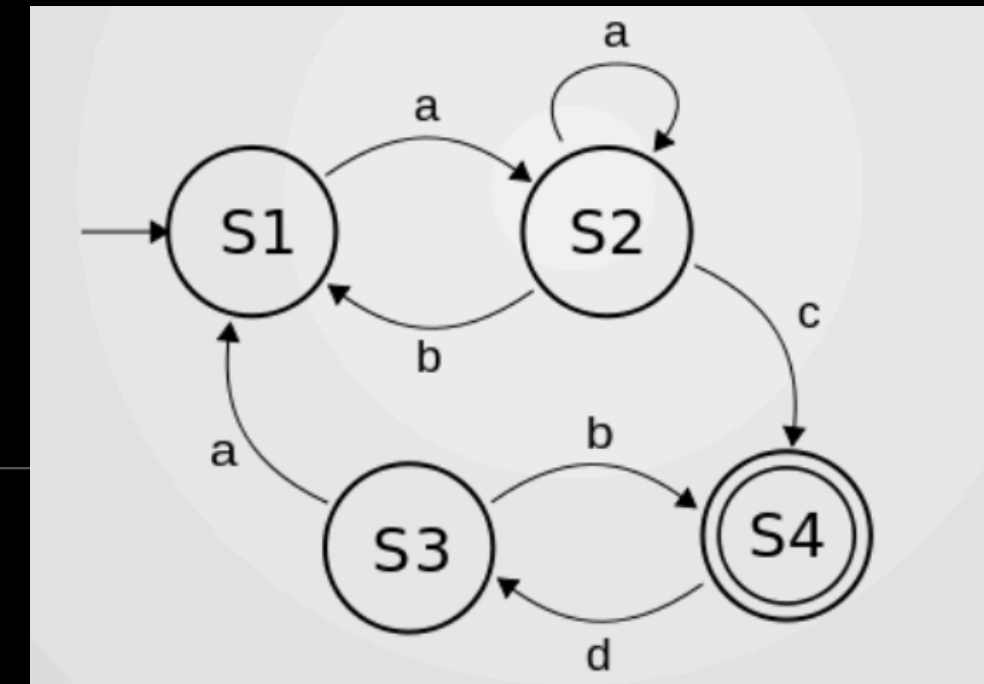
# NODO CONTROLER

## Protección ante la saturación

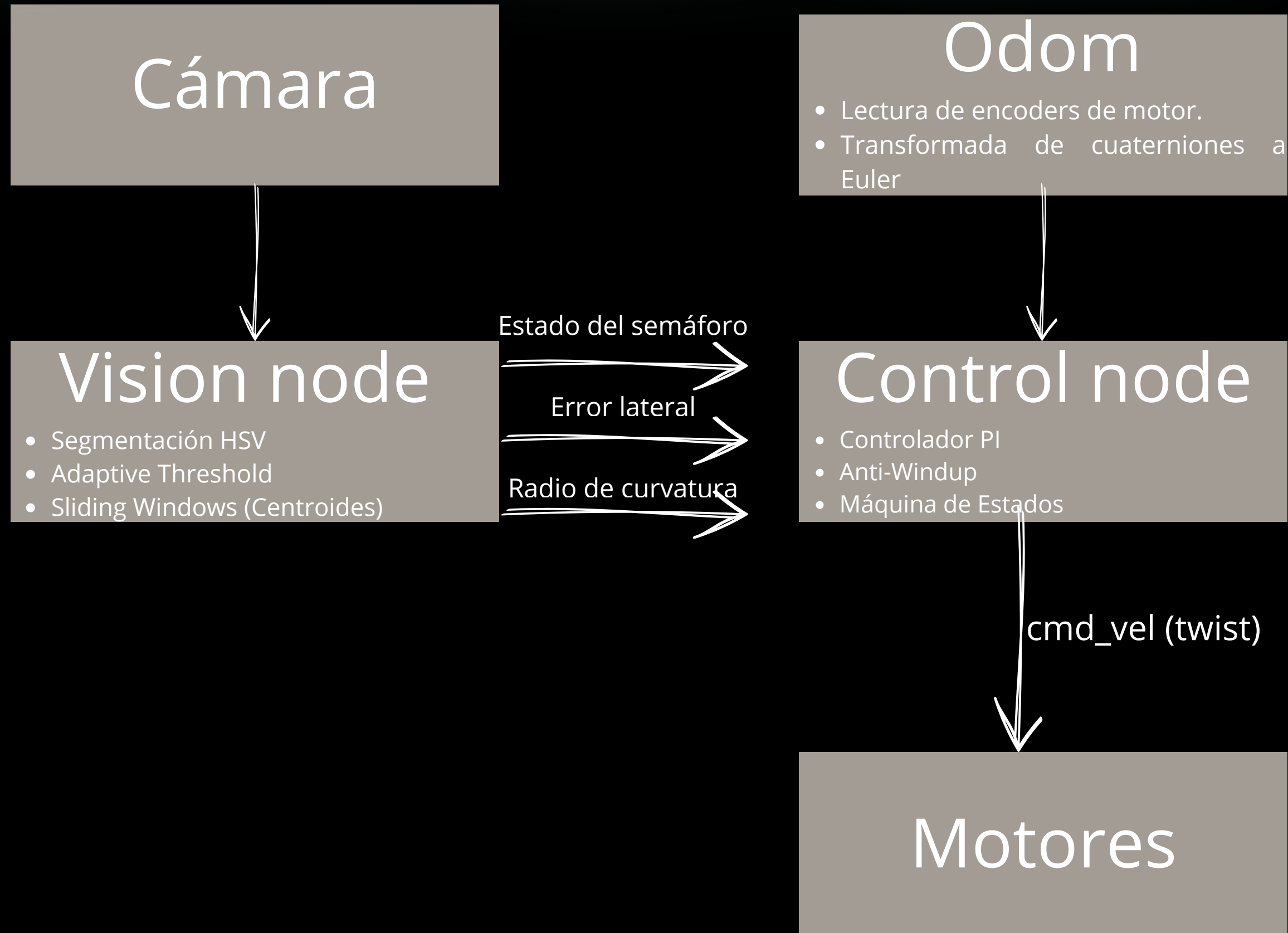
- Saturación: Limitamos la velocidad angular a 0.8 rad/s. Esto previene tirones bruscos de corriente.
- Estabilidad: Reducimos la velocidad lineal en curvas (radio\_curvatura) para bajar el centro de masa dinámico y evitar derrapes.

## Semaforo y maquina de estados

- DETENIDO: Prioridad absoluta. Si el semáforo es Rojo (0), el robot frena y limpia la memoria integral para no arrancar con tirones cuando cambie a verde.
- SIGUIENDO: Navegación normal usando el PI. Si el radio de la curva es pequeño, el robot frena automáticamente para no salirse.
- RECUPERANDO: El "Plan B". Si la visión falla. Usa la Odometría para guardar el ángulo donde se perdió y gira sobre su propio eje buscando la línea.

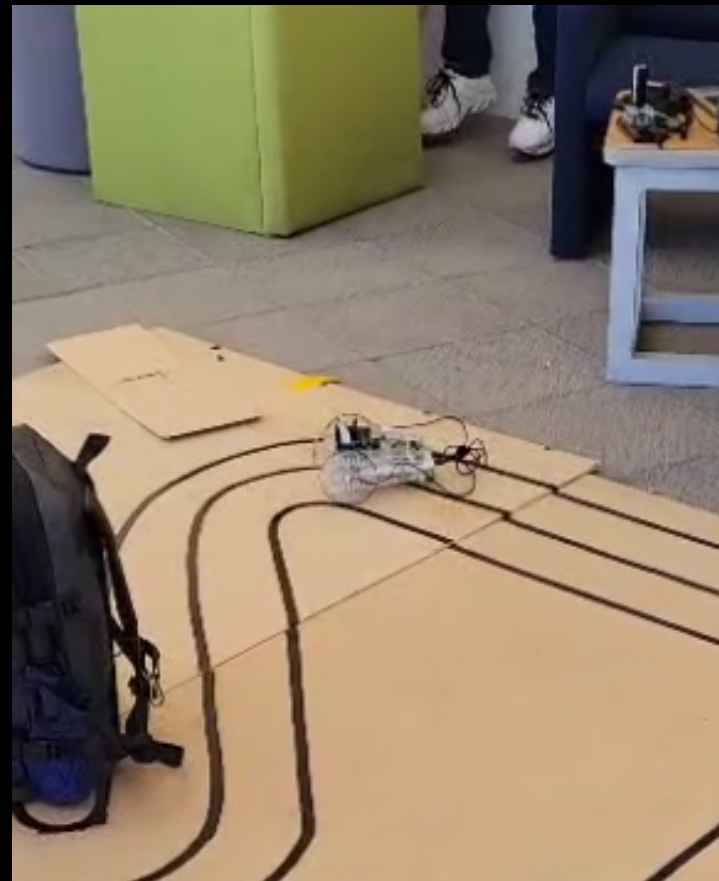


# DIAGRAMA DE FLUJO



# RESULTADOS

El sistema de navegación autónoma cumplió con los objetivos principales de diseño de manera exitosa. Se logró que el Puzzlebot realice el seguimiento de la línea con un alto grado de estabilidad y fluidez en la pista, gracias al correcto acoplamiento del algoritmo de ventanas deslizantes y el controlador PI. Ante pérdidas de ruta, el respaldo por odometría respondió con total precisión.



[Link del video de la demostracion del reto](#)



***¡Gracias Afición!***

**TE QUIERO MUCHO  
PUMAS**

